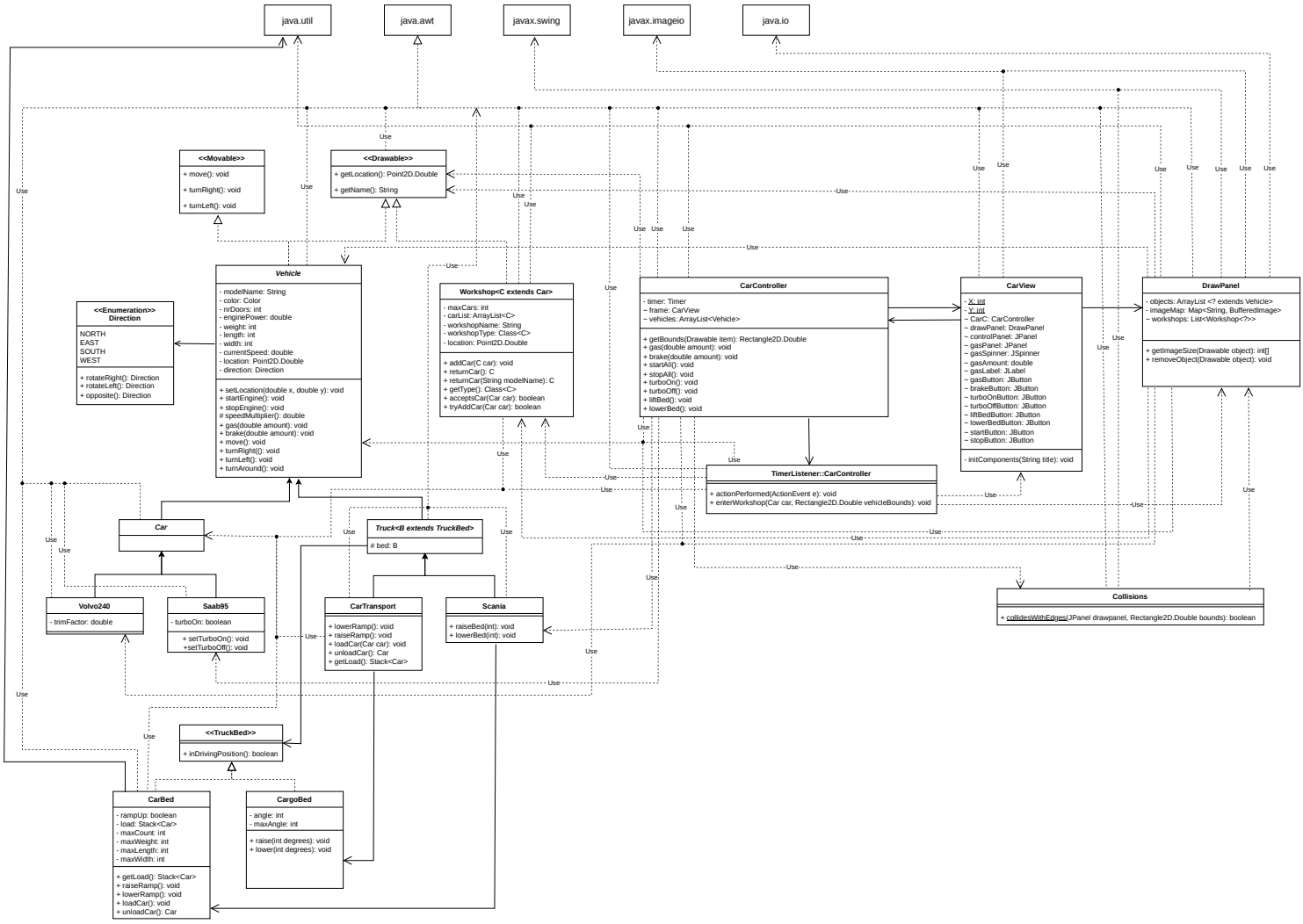
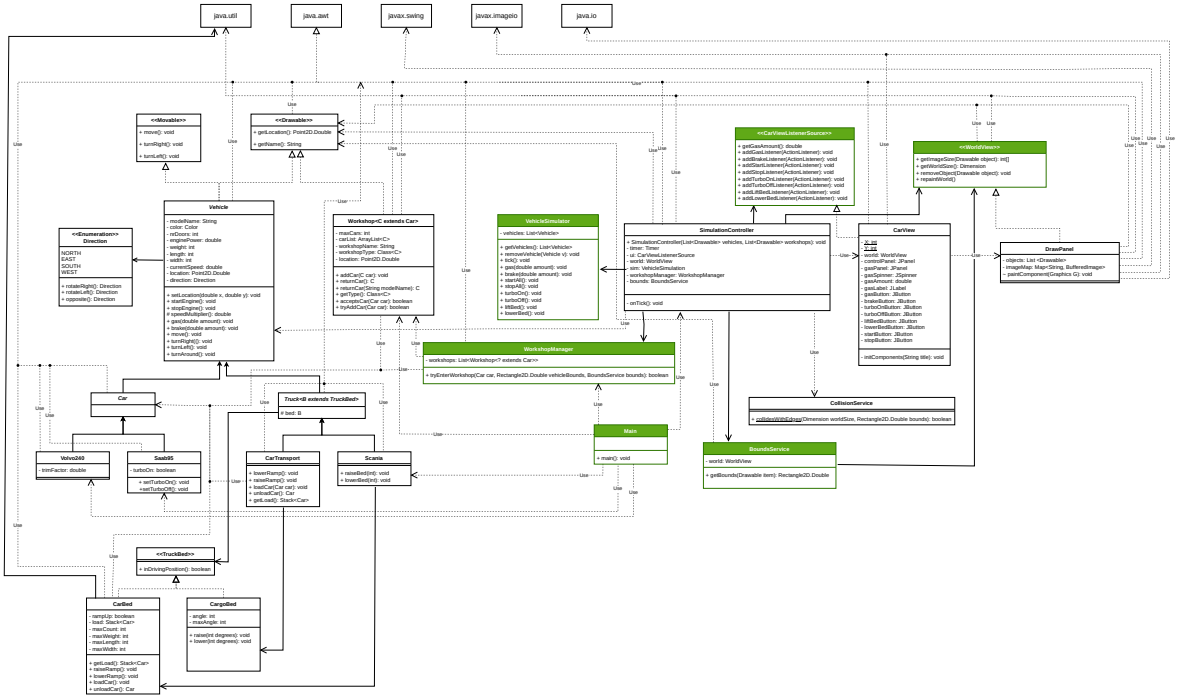

java.util
java.awt
javax.swing
javax.imageio
java.io
<<Movable>>
+ move(): void
+ turnRight(): void
+ turnLeft(): void
<<Drawable>>
+ getLocation(): Point2D.Double
+ getName(): String
Vehicle
- modelName: String
- color: Color
- nrDoors: int
- enginePower: double
- weight: int
- length: int
- width: int
- currentSpeed: double
- location: Point2D.Double
- direction: Direction
+ setLocation(double x, double y): void
+ startEngine(): void
+ stopEngine(): void
# speedMultiplier(): double
+ gas(double amount): void
+ brake(double amount): void
+ move(): void
+ turnRight((): void
+ turnLeft(): void
+ turnAround(): void
<<Enumeration>>
Direction
NORTH
EAST
SOUTH
WEST
+ rotateRight(): Direction
+ rotateLeft(): Direction
+ opposite(): Direction
Workshop<C extends Car>
- maxCars: int
- carList: ArrayList<C>
- workshopName: String
- workshopType: Class<C>
- location: Point2D.Double
+ addCar(C car): void
+ returnCar(): C
+ returnCar(String modelName): C
+ getType(): Class<C>
+ acceptsCar(Car car): boolean
+ tryAddCar(Car car): boolean
CarController
- timer: Timer
– frame: CarView
– vehicles: ArrayList<Vehicle>
+ getBounds(Drawable item): Rectangle2D.Double
+ gas(double amount): void
+ brake(double amount): void
+ startAll(): void
+ stopAll(): void
+ turboOn(): void
+ turboOff(): void
+ liftBed(): void
+ lowerBed(): void
CarView
- X: int
- Y: int
– CarC: CarController
– drawPanel: DrawPanel
– controlPanel: JPanel
– gasPanel: JPanel
– gasSpinner: JSpinner
– gasAmount: double
– gasLabel: JLabel
– gasButton: JButton
– brakeButton: JButton
– turboOnButton: JButton
– turboOffButton: JButton
– liftBedButton: JButton
– lowerBedButton: JButton
– startButton: JButton
– stopButton: JButton
- initComponents(String title): void
DrawPanel
- objects: ArrayList <? extends Vehicle>
- imageMap: Map<String, BufferedImage>
– workshops: List<Workshop<?>>
+ getImageSize(Drawable object): int[]
+ removeObject(Drawable object): void
TimerListener::CarController
+ actionPerformed(ActionEvent e): void
+ enterWorkshop(Car car, Rectangle2D.Double vehicleBounds): void
Car
Truck<B extends TruckBed>
# bed: B
Volvo240
- trimFactor: double
Saab95
- turboOn: boolean
+ setTurboOn(): void
+setTurboOff(): void
CarTransport
+ lowerRamp(): void
+ raiseRamp(): void
+ loadCar(Car car): void
+ unloadCar(): Car
+ getLoad(): Stack<Car>
Scania
+ raiseBed(int): void
+ lowerBed(int): void
Collisions
+ collidesWithEdges(JPanel drawpanel, Rectangle2D.Double bounds): boolean
<<TruckBed>>
+ inDrivingPosition(): boolean
CarBed
- rampUp: boolean
- load: Stack<Car>
- maxCount: int
- maxWeight: int
- maxLength: int
- maxWidth: int
+ getLoad(): Stack<Car>
+ raiseRamp(): void
+ lowerRamp(): void
+ loadCar(): void
+ unloadCar(): Car
CargoBed
- angle: int
- maxAngle: int
+ raise(int degrees): void
+ lower(int degrees): void
Use


java.util
java.awt
javax.swing
javax.imageio
java.io
<<Movable>>
+ move(): void
+ turnRight(): void
+ turnLeft(): void
<<Drawable>>
+ getLocation(): Point2D.Double
+ getName(): String
<<CarViewListenerSource>>
+ getGasAmount(): double
+ addGasListener(ActionListener): void
+ addBrakeListener(ActionListener): void
+ addStartListener(ActionListener): void
+ addStopListener(ActionListener): void
+ addTurboOnListener(ActionListener): void
+ addTurboOffListener(ActionListener): void
+ addLiftBedListener(ActionListener): void
+ addLowerBedListener(ActionListener): void
<<WorldView>>
+ getImageSize(Drawable object): int[]
+ getWorldSize(): Dimension
+ removeObject(Drawable object): void
+ repaintWorld()
Vehicle
- modelName: String
- color: Color
- noDoors: int
- enginePower: double
- weight: int
- length: int
- width: int
- currentSpeed: double
- location: Point2D.Double
- direction: Direction
+ setLocation(double x, double y): void
+ startEngine(): void
+ stopEngine(): void
+ getGasAmount(): double
+ gas(double amount): void
+ brake(double amount): void
+ move(): void
+ turnRight(): void
+ turnLeft(): void
+ turnAround(): void
<<Enumeration>>
Direction
NORTH
EAST
SOUTH
WEST
+ rotateRight(): Direction
+ rotateLeft(): Direction
+ opposite(): Direction
Workshop<C extends Car>
- maxCars: int
- carList: ArrayList<C>
- workshopName: String
- workshopType: Class<C>
- location: Point2D.Double
+ addCar(C car): void
+ returnCar(): C
+ returnCar(String modelName): C
+ getType(): Class<C>
+ acceptsCar(Car car): boolean
+ tryAddCar(Car car): boolean
VehicleSimulator
- vehicles: List<Vehicle>
+ getVehicles(): List<Vehicle>
+ removeVehicle(Vehicle v): void
+ tick(): void
+ gas(double amount): void
+ brake(double amount): void
+ startAll(): void
+ stopAll(): void
+ turboOn(): void
+ turboOff(): void
+ liftBed(): void
+ lowerBed(): void
SimulationController
+ SimulationController(List<Drawable> vehicles, List<Drawable> workshops): void
- timer: Timer
- ui: CarViewListenerSource
- world: WorldView
- sim: VehicleSimulation
- workshopManager: WorkshopManager
- bounds: BoundsService
- onTick(): void
CarView
- W: int
- H: int
- world: WorldView
- controlPanel: JPanel
- gasPanel: JPanel
- gasSpinner: JSpinner
- gasAmount: double
- gasLabel: JLabel
- gasButton: JButton
- brakeButton: JButton
- turboOnButton: JButton
- turboOffButton: JButton
- liftBedButton: JButton
- lowerBedButton: JButton
- startButton: JButton
- stopButton: JButton
- initComponents(String title): void
DrawPanel
- objects: List <Drawable>
- imageMap: Map<String, BufferedImage>
- paintComponent(Graphics G): void
WorkshopManager
- workshops: List<Workshop<? extends Car>>
+ tryEnterWorkshop(Car car, Rectangle2D.Double vehicleBounds, BoundsService bounds): boolean
Car
Truck<B extends TruckBed>
# bed: B
CollisionService
+ collidesWith(Edges[]Dimension worldSize, Rectangle2D.Double bounds): boolean
Main
+ main(): void
BoundsService
- world: WorldView
+ getBounds(Drawable item): Rectangle2D.Double
Volvo240
- trimFactor: double
Saab95
- turboOn: boolean
+ setTurboOn(): void
+setTurboOff(): void
CarTransport
+ lowerRamp(): void
+ raiseRamp(): void
+ loadCar(Car car): void
+ unloadCar(): Car
+ getLoad(): Stack<Car>
Scania
+ raiseBed(int): void
+ lowerBed(int): void
<<TruckBed>>
+ inDrivingPosition(): boolean
CarBed
- rampUp: boolean
- load: Stack<Car>
- maxCount: int
- maxWeight: int
- maxLength: int
- maxWidth: int
+ getLoad(): Stack<Car>
+ raiseRamp(): void
+ lowerRamp(): void
+ loadCar(): void
+ unloadCar(): Car
CargoBed
- angle: int
- maxAngle: int
+ raise(int degrees): void
+ lower(int degrees): void
Use